Source: docs\INVENTORY.md | Export: 11 November 2026

Inventory administration

`/admin/inventory` uses the approved compact admin shell, local Lora font, maroon/ivory palette, desktop table and mobile cards. Components and pure helpers live in `src/admin/inventory/`. Stock adjustment still uses the trusted `inventory:adjust` operation in `server/business.ts`.

Scope and quantities

The overview loads the complete authenticated catalogue through existing paginated `list` requests, bounded to 201 pages / 10,000 products. It does not publish partial totals or enable export if any page fails. Summary cards ignore filters and include all publication states. Available Sarees is the sum of simple quantities or individual colour quantities once. Low-stock Items counts only positive stock at or below the product threshold; Out-of-stock Items counts zero stock sources. Publication and category visibility remain separate from stock condition.

Search includes product names/codes and colour names/codes. Category, stock, structure and publication filters apply to products across the complete loaded catalogue; an expanded matching product shows all its colours for context. Sorting and 10-product pagination follow filtering. A parent colour-product quantity is its combined total, never an extra stock source. Timestamps are explicitly product timestamps; no per-colour update time is inferred. Refresh reads the catalogue again. Established administrator change signals refresh inventory after writes, including changes from another admin tab, without resetting an open adjustment draft.

Adjustments and safety

Opening an adjustment first reads the latest product and revision. Add, Remove and Set Final Quantity calculate a signed delta, with a before/change/after review followed by explicit confirmation. Supported quantities and the product-wide threshold are whole numbers from 0 through 1,000,000. A reason is required; notes are optional (500 characters). The threshold applies to every colour; mode changes remain in the Product editor. To change only a threshold, Add 0 and change the threshold. No-op requests are rejected.

The existing transaction checks enabled administrator access, current revision and stable colour identity. The catalogue entry point also verifies the `admin: true` claim. The transaction updates only inventory when quantity changes, the threshold, revision and update timestamp. It atomically creates one audit receipt containing actor, reason, notes, previous/resulting inventory, selected colour, signed delta, thresholds and server timestamp. Threshold-only events explicitly have source `threshold` and delta 0. Prices, media, publication, legacy fields and order records are preserved.

The same exact payload retains its operation ID across a failed/lost response. Replays return the existing result without another quantity change or audit; changed payloads cannot reuse that ID. A revision conflict keeps inputs and requires loading latest stock and reviewing again. An Add/Remove keeps the entered units; Set Final keeps the entered target. A conflict never silently recalculates and submits. Dirty close/navigation asks for confirmation; focus/window changes do not. Sign-out/unmount aborts reads and clears private UI state.

Existing checkout deduction, cancellation restoration and inspected-return restoration are unchanged. A manual reason mentioning a return does not update a return/order or trigger automatic restoration; use order management for that workflow.

History and export

History reads the existing `inventoryAudit` ledger only, newest first, through 25-event API pages. It completes a bounded read (401 pages / 10,000 events) before showing filters, counts or 10-event UI pages. Product/colour and order versus manual/editor filters apply to that completed result. Before/after values and signed deltas come only from recorded snapshots. Order-linked events link to the admin order route; timestamps use Asia/Kolkata. Older incomplete records show “Not recorded” and never invent actor names, quantities, times or source labels. No history is backfilled or rewritten. Threshold events remain product-wide when a colour scope is selected.

CSV exports matching results, the entire loaded catalogue, or explicitly selected matching products. Each simple or colour row appears once. CSV cells use the established escaping and spreadsheet-formula protection. Export cannot turn a partial read into success. Reserved/incoming quantities, barcode, certifications and warehouse fields are omitted because they are not authoritative fields in this application. XLSX, bulk threshold writes and prototype simulators are omitted; no import or bulk stock overwrite is introduced.

Local verification and release

- Unit coverage: `src/admin/inventory/model.test.ts`, included in `npm test`.
- Trusted API/emulator coverage: `scripts/inventory-emulator.test.ts` (also in `npm run test:admin`).
- Browser coverage: `scripts/inventory-browser.test.py`, in the existing admin emulator harness. Its fixture helper requires both exact loopback emulator hosts before writing synthetic audit records.
- Typecheck/build: `npm run lint`, `npm run build`.

Accept locally with emulators: open `/admin/inventory`, search/filter, expand a colour product, adjust with review/confirmation, test conflict/retry, inspect History and export CSV. Use only emulator fixtures for mutations.

Release requires application deployment by the owner. No new collections, Firebase rules, index definitions, dependencies, migration or production preparation are needed. History reuses the existing COLLECTION index on `inventoryAudit`: `productId ASCENDING`, `createdAt DESCENDING` (implicit document name descending). Its production readiness was not queried during this local UI task. Local emulator success does not prove hosted readiness. No production records, media, accounts, rules or indexes were changed.